SEN3244 Software Architecture

Lecture Materials: Wednesday, May 13, 2026

Lecturer: Engr. Tekoh Palma

Self-Learning Exercise — 24-Hour Intensive

Introduction

In line with our Self Learning Approach to Learning, in the previous practical you learnt to package a single web application with Docker and ran it locally with Docker Compose. Real production systems however, are not single applications — they are fleets of cooperating microservices that communicate through events, scale independently, and must remain available even when individual components fail.

This practical asks you to build and operate a realistic, event-driven, Microservice Emergency management Application called **Helep**. It is a 24-hour individual sprint that exercises topics covered in the course and every topic covered in the Kubernetes lecture video (TechWorld with Nana — *Kubernetes Tutorial for Beginners [FULL COURSE in 4 Hours]*).

You will receive a working set of **Python microservices** that implement the SRS for the *Incidence/Emergency Response Platform* (**HELEP**). The services already speak to each other through an **Apache Kafka** event bus (`aiokafka` client). Your job is to **containerize, orchestrate, package, secure, observe, and ship** the platform on Kubernetes — and to produce a short **5-minute video** demonstrating the orchestration of your application and a **source code folder** containing all your artifacts (Dockerfiles, config files, etc.). The folder should follow this naming format:

- `<matricule>-<student's name>-helep.zip`

Submissions should be done using this Form:

- `https://forms.gle/9QCvLTMV3CSZpxPc8`

Resources you are working from:

- **SRS:** `Use Case Emergency incidence response platform(Helep).docx`
- **Source code:** `helep/services/` (5 services, FastAPI)

- **Templates:** `helep/design-process-template.md`, `helep/patterns-template.md`
- **Local-dev only:** `helep/docker-compose.dev.yml` (NOT a deliverable — exists to let you verify the saga end-to-end before you start writing Kubernetes manifests)

Lecture Video

You should already have watched the K8s lecture video. Re-watch the chapters on **StatefulSet, ConfigMap/Secret, Volumes, Ingress, and Helm** if you are unsure — these account for the largest share of marks.

- `https://youtu.be/X48VuDVv0do`

Task

Within **24 contiguous hours** starting from `8am Wednesday, May 13, 2026 - 8am Wednesday, May 14, 2026` produce a working Kubernetes deployment of HELEP plus the supporting artefacts listed under *Deliverables*.

Learning Objectives

At the end of this exercise you should be able to:

- Implement the architectural design process (Lecture 4) to a real SRS
- Recognise and apply software design patterns in a microservice context (Lecture 3)
- Containerize a polyglot-friendly Python microservice with a multi-stage Dockerfile
- Author Kubernetes manifests for Deployment, Service, Ingress, ConfigMap, Secret, PVC, StatefulSet, HPA, NetworkPolicy
- Package a multi-service application as a Helm umbrella chart
- Bootstrap a CI/CD pipeline (Jenkins) that builds, scans, pushes, and deploys
- Stand up minimum observability (Prometheus + Grafana + dashboard)
- Apply baseline DevSecOps practices (image scan, NetworkPolicy default-deny, Secret hygiene)

What is provided vs what you must build

Provided (lecturer)	You build
5 Python services with FastAPI, structlog, prometheus-client, `/healthz`, `/readyz`, `/metrics`, Kafka producer + consumer-group reader (`aiokafka`)	One **Dockerfile per service** (multi-stage, non-root)
`architecture-overview.md` **(lecturer reference — you must re-derive equivalent content in your L4 doc)**	A **Helm umbrella chart** under `charts/helep/` with one sub-chart per service
`design-process-template.md` **skeleton**	A **K8s manifest set** rendered by the Helm templates (Deployment, Service, ConfigMap, Secret, Ingress, HPA, PVC, NetworkPolicy)
`patterns-template.md` **skeleton**	A **Kafka cluster via Strimzi Operator** (Kafka CR + KafkaTopic CRDs, persistent storage)
`docker-compose.dev.yml` **(dev-only, NOT graded)**	**Observability** — Prometheus + Grafana + a dashboard JSON file
Per-service `README.md` listing endpoints	**CI/CD** pipeline (Jenkinsfile)
	L3 patterns-in-code document (~3 pages)
	L4 design-process document (~5 pages)
	5-min demo video

The system you are deploying (1-paragraph summary)

HELEP receives an **SOS** from a citizen, **dispatches the nearest free responder**, **notifies** both parties, and feeds a **police analytics view** with incident counts per zone. All this happens via events on **Apache Kafka** (single broker for local dev, Strimzi-managed in K8s) using topics + consumer groups, **partitioned by `incident_id`** so all events for one SOS stay ordered on a single partition. Five services are involved: `user-service`, `sos-service`, `dispatch-service`, `notification-service`, `analytics-service`. The saga is choreographed — there is no central orchestrator.

Detailed event flow, ports, and SRS mapping are in your own Lecture 4(L4) design-process doc (you derive it; the lecturer reference is `architecture-overview.md`). i.e use the teachers document to come up with your equivalent `architecture-overview` following the architectural design process.

Practical Exercise — Deliverables

Part A — Containerization (10%)

For each of the 5 services, author a **Dockerfile** that:

- Uses a multi-stage build (final image based on `python:3.12-slim` or `python:3.12-alpine`).
- Runs the app as a **non-root** user (UID ≥ 10001).
- Exposes only the service port (8001–8005).
- Copies only `app/` and `requirements.txt` into the final stage.
- Passes `trivy image --severity HIGH,CRITICAL` with zero findings (a clean baseline image is acceptable; document any whitelisted CVEs).

Push images to your own Docker Hub namespace (e.g. `dockerhub.io/<you>/helep-<svc>:0.1.0`).

Part B — Helm Umbrella Chart (20%)

Create `charts/helep/` as a Helm **umbrella chart** that contains one sub-chart per service. Kafka is managed by **Strimzi** (Part D) and is not part of this umbrella — declare it as a runtime dependency only (your services reference the bootstrap service `<kafka-cluster>-kafka-bootstrap.kafka.svc.cluster.local:9092`).

Minimum templates required per service sub-chart:

- `Deployment` (with `livenessProbe` + `readinessProbe` pointing at `/healthz` + `/readyz`)
- `Service` (ClusterIP)
- `ConfigMap` for non-secret env (`KAFKA_BOOTSTRAP`, `MATCHER`, …)
- `Secret` for `JWT_SECRET`
- `HorizontalPodAutoscaler` (CPU target 70 %, min 1 / max 3)
- `PersistentVolumeClaim` for `/data` (so SQLite files survive pod restarts)

Plus an umbrella `Ingress` exposing `user-service`, `sos-service`, and `analytics-service` (e.g. on `helep.local`).

The values file (`values.yaml`) must parameterise: image repo + tag, replica count, ingress host, Kafka bootstrap server, and `MATCHER` strategy.

Part C — Kubernetes manifest extras (10%)

Beyond what Helm generates, deliver:

- A `**NetworkPolicy**` per service with default-deny ingress + explicit allow for Ingress controller, Prometheus scraper, and Kafka brokers in the `kafka` namespace.
- A dedicated `**Namespace**` `helep` (everything except observability lives here).
- An `**Ingress**` resource (NGINX or Traefik — your choice) routing `/api/users/*`, `/api/sos/*`, `/api/stats/*` to the right backends.
- A `**StorageClass**` declaration (you can rely on the cluster default, but cite which class is used and why).

Part D — Kafka via Strimzi Operator (5%)

Install **Strimzi** (Helm: `oci://quay.io/strimzi-helm/strimzi-kafka-operator`) into a dedicated `kafka` namespace, then deploy:

- A `Kafka` custom resource named `helep-kafka` (one broker + one controller acceptable for the 24 h time frame — **explicitly cite this limitation** in your design doc; production requires ≥ 3 brokers + ≥ 3 controllers for replication and quorum).
- **Persistent storage** via `volumeClaimTemplate` in the Kafka CR (≥ 1 Gi).
- A `KafkaTopic` CRD per helep event (7 total):
- `user.registered`, `sos.triggered`, `sos.cancelled`, `responder.assigned`,
- `responder.confirmed`, `safety.zone.entered`, `notification.sent`
- Each topic: `partitions: 3`, `replicas: 1` (raise to 3 if running ≥ 3 brokers).
- A `PodDisruptionBudget` on the broker StatefulSet (`minAvailable: 1`).
- Disable broker auto-create (set `auto.create.topics.enable: false` in the Kafka CR `config`) — topics MUST exist via `KafkaTopic` CRDs.
- All helep services point at `KAFKA_BOOTSTRAP=helep-kafka-kafka-bootstrap.kafka.svc.cluster.local:9092` (or your chosen cluster name).

Part E — Observability (10%)

- Install `kube-prometheus-stack` (Helm) into namespace `observability`.
- Annotate each helep service so Prometheus scrapes `/metrics`.
- Provide a **dashboard JSON** (`dashboards/helep.json`) with at minimum:
- SOS triggers per second (by mode)
- Notifications delivered per template
- Dispatch latency (assignment count over time)

Part F — CI/CD (10%)

Author a **Jenkinsfile** declarative pipeline (in `ci/Jenkinsfile`). GitHub Actions and other CI systems are not accepted for this exercise.

Stages required:

1. Lint Python (`ruff check` or `flake8`)
2. Build Docker images (one per service)
3. Scan images with **Trivy**, fail on HIGH+
4. Push to your registry
5. `helm upgrade --install helep ./charts/helep -n helep` against your cluster
6. Run a smoke test against `/healthz` of each service
7. Manual approval gate before "prod" namespace deploy (Jenkins `input` step)

Part G — Design Process Document (15%)

Complete `design-process-template.md` and submit as `design.pdf`. Approximately 5 pages, ~2000–3000 words. Must include all required steps in the architectural design process.

Part H — Patterns-in-Code Document (10%)

Complete `patterns-template.md` and submit as `patterns.pdf`. Approximately 3 pages. **Code citations are required** — `file:line` for every pattern you claim.

Part I — Demo Video (10%)

A **5-minute screen recording** showing:

8. `kubectl get all -n helep` and `kubectl get pv,pvc -n helep`
9. Trigger a full SOS via `curl` and tail `notification-service` logs
10. Cancel an SOS — show the responder released
11. **Chaos**: `kubectl delete pod -l app=dispatch-service` and prove the saga still works (Kafka consumer group rebalances, un-committed offsets re-deliver).
12. Grafana dashboard with non-zero metrics
13. CI/CD pipeline final green run

Grading Rubric (100 points)

Part	Weight	Pass bar
A — Containerization	10	All 5 images build + non-root + Trivy clean
B — Helm chart	20	`helm upgrade --install` succeeds on a clean cluster
C — K8s extras	10	NetworkPolicies enforced (prove by `kubectl exec` deny test)
D — Kafka via Strimzi	5	`Kafka` + `KafkaTopic` CRs deploy, pod restart, messages survive
E — Observability	10	Dashboard shows live

		metrics during demo
F — CI/CD	10	End-to-end green pipeline including Trivy gate
G — Design doc	15	Every section completed + traceability shown
H — Patterns doc	10	Every claim has a `file:line` citation
I — Demo video	10	Saga + chaos + cancel all visible
Bonus — L6 extension	+5	Add a 6th service: AI credibility scorer (call out-of-process model OR a tiny scikit-learn classifier) — must wire into events

Mapping to course outline (so you know each lecture is graded)

Lecture	Where it shows up
L2 — Architectural style justification	inside *Design doc* (G)
L3 — Patterns & principles	dedicated *Patterns doc* (H)
L4 — Architectural design process	*Design doc* structure (G)
L5 — DevOps & DevSecOps	Parts A–F
L6 — Future of architecture	optional bonus

Mapping to the Kubernetes lecture video

Nana chapter	Where assessed
Pod, Deployment	Part B
Service (ClusterIP, NodePort, LoadBalancer, Headless)	Parts B + D
Ingress	Part C
ConfigMap, Secret	Part B
Volumes (PV / PVC / StorageClass)	Part C + D
StatefulSet	Part D
Namespaces	Part C
Helm	Part B
kubectl commands	demo video (I)
YAML manifests	every Part

Allowed tools

- Local Kubernetes: **Minikube**, **Kind**, **k3d**, or **Docker Desktop K8s** — your choice.
- Helm 3.
- Docker Hub or GHCR for image storage.
- Jenkins for CI/CD (mandatory — no alternatives).
- Trivy for image scan.
- Prometheus + Grafana for observability (any installation method).

Forbidden:

- Generative-AI-authored Dockerfiles / manifests **without proof of understanding** (you will be asked to defend any file at viva).
- Pre-built Helm charts copied verbatim from public repos — your chart must be authored by you.

Submission

Submit a **single zip** named `<matricule>-<student's name>-helep.zip` containing:

```
<matricule>-<student's name>-helep/
├── services/                # the lecturer-provided code, unmodified
├── charts/helep/            # your Helm umbrella chart
├── manifests/               # any standalone manifests not in Helm (e.g.
raw NetworkPolicies)
├── ci/                      # Jenkinsfile
├── dashboards/helep.json    # Grafana dashboard JSON
├── design.pdf               # Part G
├── patterns.pdf             # Part H
└── demo.mp4 OR demo-link.txt  # Part I (≤ 5 min, ≤ 200 MB; if larger,
upload to Drive and submit the link)
```

Submission form: `https://forms.gle/9QCvLTMV3CSZpxPc8`

Deadline: `8am Wednesday, May 14, 2026`

Late submissions deducted at 10 points / hour (cap 50 points).

Academic Integrity

You may discuss approaches with classmates but **all code, manifests, and writing must be your own**. Image hashes and commit history will be inspected. Plagiarism, including verbatim copy of Helm charts, manifests, or design docs from public sources, is a zero on the entire assignment.

Good Luck My Belove Engineers – Engr. Tekoh Palma